Comprehensive FYP Feasibility Analysis

AI-Driven Interview and Technical Assessment Platform

---

1. Overall Feasibility

Rating: Medium to Low

Justification:

This project is **overly ambitious** for a 3-person undergraduate team within one academic year under a zero-budget constraint. While the concept is educationally valuable and addresses a real problem, the implementation scope encompasses multiple complex subsystems that typically require specialized expertise and significant computational resources.

Key Concerns:

- **Scope Creep**: 10 distinct modules, each representing a substantial project on its own
- **Integration Complexity**: Real-time voice, NLP, computer vision, code execution, and behavioral analysis must work seamlessly together
- **AI/ML Workload**: Multiple AI components requiring different expertise (NLP, CV, speech processing)
- **Security Risks**: Code execution sandboxing is non-trivial and dangerous if done incorrectly
- **Quality vs. Timeline**: Achieving "production-ready" quality across all components in 12 months is unrealistic

Realistic Assessment: As proposed, this project has a **30-40% completion probability** with acceptable quality standards. With significant scope reduction, this could increase to **70-80%**.

---

2. Technical Complexity & Risk Areas

High-Risk Components:

2.1 Code Execution Module (CRITICAL RISK)

- **Complexity**: Creating a secure sandbox for arbitrary code execution

- **Security Concerns**: Remote code execution vulnerabilities, resource exhaustion attacks, infinite loops
- **Implementation Challenge**: Professional platforms (LeetCode, HackerRank) have dedicated security teams for this
- **Undergraduate Reality**: Students may not have systems programming or cybersecurity expertise
- **Risk Level**: ⚠️ **VERY HIGH** - Could compromise entire system security

2.2 Real-Time Multimodal AI Processing

- **Challenge**: Simultaneous processing of voice, video, and text in real-time
- **Latency Requirements**: Interview flow requires < 2-3 second response times
- **Resource Intensity**: Running multiple AI models concurrently (STT, TTS, CV, NLP)
- **Risk Level**: ⚠️ **HIGH** - Performance bottlenecks likely

2.3 Computer Vision Behavioral Analysis

- **Technical Depth**: Eye tracking, gaze detection, facial analysis
- **Model Requirements**: Requires trained models for multiple indicators
- **Accuracy Challenges**: Lighting conditions, camera quality, head pose variations
- **Ethical Concerns**: Privacy implications and bias in behavioral assessment
- **Risk Level**: ⚠️ **HIGH** - May not achieve acceptable accuracy

2.4 Resume Parsing and Skill Extraction

- **Challenge**: Handling diverse resume formats (PDF, DOCX, images)
- **NLP Complexity**: Named entity recognition, skill taxonomy mapping
- **Data Quality**: Resumes vary wildly in structure and formatting
- **Risk Level**: ⚠️ **MEDIUM-HIGH** - Time-consuming edge case handling

2.5 Dynamic Question Generation

- **AI Requirement**: Context-aware question generation based on CV and role
- **Quality Control**: Ensuring questions are relevant, unambiguous, and appropriate difficulty
- **LLM Dependency**: May require LLM integration (raises API cost concerns)
- **Risk Level**: ⚠️ **MEDIUM** - Quality may be inconsistent without LLMs

Medium-Risk Components:

- **Speech-to-Text/Text-to-Speech**: Dependence on quality of free models
- **Answer Evaluation**: NLP semantic similarity is challenging for technical content
- **Multi-database Architecture**: PostgreSQL + MongoDB adds operational complexity
- **Audio Analysis**: Speech pattern detection requires signal processing expertise

---

3. Dataset Requirement Analysis

Datasets ARE Required

This project has **extensive dataset needs** across multiple domains:

3.1 Resume Parsing & Skill Extraction

Required:

- Annotated resume datasets with labeled skills, experience, education
- Skill taxonomy/ontology datasets

Free Sources:

- **Kaggle**: "Resume Dataset" (limited, ~1000 resumes)
- **GitHub**: Open-source resume parsers often include sample datasets
- **Indeed/LinkedIn Scraping**: Legally questionable, not recommended
- **Custom Creation**: Team may need to collect/annotate 200-500 resumes

Challenges:

- Most resume datasets are proprietary
- Manual annotation is time-intensive
- Privacy concerns with real resumes

3.2 Interview Question-Answer Pairs

Required:

- Domain-specific Q&A datasets (software engineering, data science, etc.)
- Coding problem datasets with solutions

Free Sources:

- **GitHub**: Awesome Interview Questions repositories
- **LeetCode/HackerRank**: Scraping is against ToS, but problem descriptions are public
- **StackOverflow**: Developer Survey and Q&A dumps
- **Kaggle**: Job interview question datasets
- **Academic**: Microsoft CodeXGLUE, APPS (Automated Programming Progress Standard)

Quality Assessment: Good availability, but curation and difficulty tagging will require manual effort

3.3 Behavioral Analysis Training Data

Required:

- Face datasets for gaze detection, head pose estimation
- Speech datasets for confidence/hesitation analysis

Free Sources:

- **MPIIGaze**: Eye gaze estimation dataset
- **AFLW2000-3D**: Face alignment dataset
- **CommonVoice by Mozilla**: Speech dataset (general purpose)
- **YouTube Face Database**: For facial analysis

Challenges:

- Interview-specific behavioral datasets don't exist publicly
- Team would need to collect custom data (ethical approval required)
- Bias in facial analysis models is well-documented

3.4 Code Correctness Datasets

Required:

- Programming problems with test cases
- Code submissions (correct and incorrect)

Free Sources:

- **Kaggle**: Code similarity and bug detection datasets
- **GitHub**: CodeSearchNet, The Stack
- **Academic**: CodeNet (IBM), CodeContests (Google DeepMind)

Quality Assessment: Excellent availability, well-curated

Dataset Feasibility Summary:

Dataset Type	Availability	Quality	Effort to Prepare
Resumes	Limited	Variable	High (annotation)
Interview Q&A	Good	Medium-High	Medium (curation)
Coding Problems	Excellent	High	Low
Behavioral Data	Poor	N/A	Very High (collection)

Critical Gap: Behavioral analysis datasets specific to interview contexts are **not publicly available**. Team would need institutional ethics approval to collect video/audio of mock interviews.

---

4. AI / ML / LLM Usage

AI/ML Components Breakdown:

4.1 Natural Language Processing

Use Cases:

- Resume parsing (NER, text extraction)
- Answer relevance analysis
- Question generation
- Skill matching

Models Needed:

- **Pretrained BERT/RoBERTa**: For semantic similarity (HuggingFace)
- **spaCy NER models**: For resume parsing
- **Sentence Transformers**: For answer evaluation

Training Required:

- **Fine-tuning**: Yes, for domain-specific answer evaluation (~10-20 hours on GPU)
- **From Scratch**: No

Open-Source Options:

- `sentence-transformers/all-MiniLM-L6-v2` (inference only)
- `bert-base-uncased` (fine-tuning feasible)
- spaCy pretrained models (English NLP pipeline)

4.2 Computer Vision

Use Cases:

- Eye gaze tracking
- Facial landmark detection
- Head pose estimation
- Expression analysis (if included)

Models Needed:

- **MediaPipe Face Mesh**: Eye/face tracking (Google, free)
- **OpenFace**: Facial behavior analysis toolkit
- **Dlib**: Face detection and landmark prediction

Training Required:

- **Pretrained Models**: Yes, available
- **Custom Training**: Ideally yes, but not feasible given time/data constraints
- **Inference Only**: Realistic approach

Challenges:

- Real-time processing on CPU may be slow (15-20 fps achievable)
- Accuracy degrades with poor lighting/camera quality

4.3 Speech Processing

Use Cases:

- Speech-to-text conversion
- Text-to-speech synthesis
- Speech pattern analysis (pace, pauses, hesitations)

Models Needed:

- **Whisper (OpenAI)**: Open-source STT, state-of-the-art
- **Coqui TTS**: Open-source TTS
- **Wav2Vec 2.0**: Speech representation learning

Training Required:

- **STT/TTS**: Pretrained models sufficient
- **Speech Analysis**: May need fine-tuning for hesitation detection

Free Options:

- Whisper: Fully free, runs locally
- Google Speech-to-Text: Free tier (60 min/month) - **likely insufficient**
- Coqui TTS: Fully free

4.4 Large Language Models (LLMs)

Potential Use Cases:

- Dynamic question generation

- Answer evaluation with contextual understanding
- Personalized feedback generation

Critical Constraint: Project states **"No paid APIs"**

Free LLM Options:

1. **Local Open-Source LLMs:**
    - **Llama 2 (7B)**: Free, but requires 16GB+ RAM/VRAM
    - **Mistral 7B**: Similar requirements
    - **Phi-2 (2.7B)**: Smaller, can run on 8GB systems
    - **GPT-2**: Outdated but lightweight
2. **Free API Tiers (with limits):**
    - **Cohere**: 1000 calls/month free
    - **HuggingFace Inference API**: Rate-limited
    - **Google Gemini**: Limited free tier
    - **Groq**: Fast inference, free tier available

Reality Check:

- Running 7B+ models locally requires significant hardware (see Section 5)
- Free API tiers won't sustain testing/development (100s-1000s of API calls)
- Without LLMs, question generation quality will be limited to template-based approaches

Recommendation:

- Use **template-based question generation** initially
- Integrate **Phi-2 or GPT-2** for answer evaluation (feasible on student hardware)
- Avoid dependency on external LLM APIs

---

5. Hardware & Compute Requirements

Compute Intensity Analysis:

5.1 Real-Time Processing Requirements

During a single interview session, the system must simultaneously:

1. Process video stream (15-30 fps) for behavioral analysis
2. Transcribe audio in real-time (STT)
3. Generate TTS responses

4. Evaluate answers using NLP models
5. Execute user code in sandbox
6. Manage database transactions

Estimated Load:

- **CPU**: 4-6 cores minimum for parallel processing
- **RAM**: 16GB minimum (8GB for models, 4GB for code execution, 4GB OS/services)
- **GPU**: Highly beneficial but not mandatory if optimizations applied

5.2 Model Inference Requirements

Model Type	Model Size	CPU Inference	GPU Inference	RAM
BERT-base (NLP)	110M params	50-100ms/query	10-20ms	1-2GB
Whisper-base (STT)	74M params	Real-time possible	Real-time easy	2-3GB
MediaPipe Face	Lightweight	30fps on CPU	N/A	500MB
Coqui TTS	~50M params	2-3x real-time	Real-time	1-2GB
Llama 2 7B (optional)	7B params	Very slow	20-30 tokens/sec	14GB+

Total Estimated RAM: 12-16GB for full stack (without LLM), 28GB+ with LLM

5.3 Training Requirements (if applicable)

If fine-tuning models:

- **Answer Evaluation Model**: Fine-tune BERT on domain Q&A
  - **Time**: 2-4 hours on free GPU (Colab)
  - **Feasible**: Yes
- **Behavioral Models**: Train custom models on collected data
  - **Time**: 10-20 hours on GPU
  - **Feasible**: Only if data collection successful

5.4 Free GPU Platforms

Google Colab:

- **Free Tier**: 12GB RAM, Tesla K80/T4 GPU
- **Limits**: ~12 hours/session, disconnects on idle
- **Suitable For**: Model fine-tuning, experimentation
- **Not Suitable For**: Production deployment

Kaggle Notebooks:

- **Free Tier**: 16GB RAM, P100 GPU (30 hours/week)
- **Better than Colab** for longer training runs
- **Not suitable for** production

Paperspace Gradient Free Tier:

- Limited availability, frequent waitlists

5.5 Student Hardware Reality Check

Typical Student Laptop:

- Intel i5/i7 or AMD Ryzen 5/7
- 8-16GB RAM
- Integrated graphics (no dedicated GPU)

Can the project run on this?

- **Partial Yes**: With heavy optimization
- **NLP/STT/TTS**: Feasible with CPU-only inference
- **Computer Vision**: Achievable at 15-20 fps
- **LLM Integration**: **Not feasible** (7B models need 16GB+ RAM)
- **Multiple Simultaneous Interviews**: **No** (system would crash)

Recommended Minimum:

- **CPU**: 6-core (i5-12th gen / Ryzen 5 5600)
- **RAM**: 16GB
- **GPU**: Optional but highly beneficial (GTX 1660 or better)

Cost-Free Solutions:

- Deploy backend on **Google Cloud Free Tier** (f1-micro instance, limited)
- Use **Render.com** or **Railway.app** free tiers for hosting
- Run models on **HuggingFace Spaces** (free inference)

Realistic Outcome: Students will need to optimize heavily, reduce concurrent processing, and accept slower response times.

---

6. Costing & Budget Feasibility

6.1 Cost-Incurring Components

Direct Costs:

Component	Free Option	Limitation	Potential Cost (if scaled)
Cloud Hosting	Railway/Render free tier	500MB RAM, sleeps on idle	$7-20/month
Database Hosting	PostgreSQL (Supabase free)	500MB storage	$25/month
	MongoDB Atlas free	512MB storage	$9/month
GPU Compute	Colab/Kaggle	Can't deploy production	$0.50-2/hour (cloud GPUs)
Storage	GitHub (100MB limit)	Video/model storage issue	$5-10/month
LLM API Calls	Free tiers (limited)	1000-5000 calls/month	$0.01-0.10 per call
STT/TTS	Whisper/Coqui (local)	Requires compute	$0.006/min (Google STT)

Hidden Costs:

1. **Bandwidth**: Video upload/processing (free tiers: 100GB/month)
2. **CDN**: If serving model files (Cloudflare free tier exists)
3. **Email Services**: For user notifications (SendGrid free: 100 emails/day)
4. **SSL Certificates**: Let's Encrypt (free)

6.2 Budget Assessment

Can this be done at ZERO cost?

Short Answer: Yes, but with significant limitations

Free Tier Strategy:

1. **Frontend**: Deploy on **Vercel** or **Netlify** (free)
2. **Backend**: Use **Render.com** free tier (sleeps after 15 min inactivity)
3. **Database**:
    - PostgreSQL: **Supabase** (500MB free)
    - MongoDB: **MongoDB Atlas** (512MB free)
4. **AI Models**: Run locally or use **HuggingFace Inference API** (rate-limited)
5. **Code Execution**: Build custom sandbox (risky) or use **Judge0 API** (50 calls/day free)

Limitations of Free Approach:

- **Cold Starts**: 10-30 second delays when service wakes up
- **Storage Limits**: Can't store many user videos or large models
- **API Rate Limits**: Development/testing will hit limits quickly
- **No Scalability**: Only 1-2 concurrent users realistically
- **Poor User Experience**: Slow, unreliable for demos

Low-Cost Option ($10-20/month):

- Upgrade to **Railway Pro** or **Digital Ocean** ($10/month)
- **Pros**: Better reliability, faster responses, more storage
- **Cons**: Still not scalable, but acceptable for FYP demo

6.3 Cost-Free Alternatives

Paid Component	Free Alternative	Trade-off
Cloud GPU inference	Local inference on CPU	Slower (10-50x)
Managed database	SQLite (local file)	No cloud sync, single-user
LLM API	Template-based generation	Lower quality
Video storage	Process and discard	No replay/review
Code execution API	Custom Docker sandbox	Security risk

---

7. Dependency on Paid APIs or Services

7.1 Direct Dependencies (As Proposed)

Potential Paid Dependencies:

1. **LLM APIs** (for question generation/answer evaluation)
    - **Risk**: High likelihood students will want GPT-3.5/4 integration
    - **Free Alternative**: Llama 2, Mistral, Phi-2 (local), HuggingFace rate-limited API
    - **Reality**: Quality drop is significant with smaller models
2. **Speech-to-Text**
    - **Proposed**: Not specified, likely Google/Azure STT
    - **Free Alternative**: **Whisper (OpenAI)** - fully free, open-source, runs locally

    - **Verdict**: ✅ **No dependency** if using Whisper
3. **Text-to-Speech**
    - **Proposed**: Not specified
    - **Free Alternative**: **Coqui TTS, gTTS (Google TTS library)**
    - **Verdict**: ✅ **No dependency** with free options
4. **Code Execution**
    - **Risk**: Judge0, HackerRank API, etc. have very limited free tiers
    - **Free Alternative**: Custom sandbox (Docker, Kubernetes)
    - **Verdict**: ⚠️ **High-risk component** - requires systems expertise
5. **Database Hosting**
    - **Free Options**: Supabase, MongoDB Atlas, PlanetScale, Neon
    - **Verdict**: ✅ **No dependency** for prototype/FYP scale

7.2 Indirect Dependencies

Areas where students might inadvertently introduce costs:

1. **Model Hosting**: Deploying large models may require paid compute
    - **Solution**: Use HuggingFace Spaces (free) or local deployment
2. **Video Storage**: Storing interview recordings
    - **Solution**: Process and delete immediately, or use free tier limits
3. **Email Notifications**: Sending assessment reports
    - **Solution**: SendGrid free tier (100 emails/day)

7.3 Explicit Confirmation

Question: Does this project require paid APIs?

Answer:

As currently proposed: Likely YES (implicit LLM usage for question generation)

With modifications: NO - The project CAN be implemented entirely with free/open-source tools:

- ✅ Whisper for STT (free)
- ✅ Coqui TTS for TTS (free)
- ✅ Local BERT/spaCy for NLP (free)
- ✅ MediaPipe for CV (free)

- ✅ Open-source LLMs (Phi-2, Llama 2) or template-based approach
- ✅ Custom code execution sandbox (technically free but high-risk)

Recommended Architecture (Zero-Cost):

Frontend: Next.js (Vercel free hosting)
Backend: FastAPI (Render.com free tier)
Database: PostgreSQL (Supabase free) + MongoDB Atlas free
AI Models: HuggingFace Transformers (local inference)
STT: Whisper (local)
TTS: Coqui TTS (local)
Code Execution: Custom Docker sandbox (self-hosted)

Critical Warning: Students must explicitly design the system to avoid API costs from day one. It's easy to prototype with paid APIs (OpenAI, Google Cloud) and then realize at the end that deployment is unaffordable.

---

8. Risk Mitigation & Scope Control

8.1 Immediate Scope Reductions (Essential)

Priority 1: Eliminate High-Risk Components

❌ REMOVE: Real-Time Computer Vision Behavioral Analysis

- **Reason**:
  - Requires custom dataset collection (ethics approval, time-intensive)
  - Marginal academic value for FYP
  - Creates privacy/bias concerns
  - Processing overhead kills performance
- **Alternative**: Focus on audio-based indicators (pace, pauses) only
- **Impact**: Reduces 30% of technical complexity

❌ REMOVE: Multi-Database Architecture (PostgreSQL + MongoDB)

- **Reason**: Adds operational complexity without significant benefit
- **Alternative**: Use PostgreSQL with JSONB columns for flexibility
- **Impact**: Simplifies deployment and reduces learning curve

❌ REMOVE: Real-Time Voice Interaction

- **Reason**: Adds latency, complexity, and compute requirements

- **Alternative**: Text-based interview with optional audio upload for voice analysis
- **Impact**: Reduces 25% of real-time processing load

Priority 2: Simplify AI Components

Question Generation:

- ❌ **Remove**: LLM-based dynamic generation
- ✅ **Replace**: Rule-based templates + skill matching from resume
- **Example**: "You mentioned {skill} in your resume. Explain {predefined_question_template}."

Answer Evaluation:

- ❌ **Remove**: Deep semantic understanding with LLMs
- ✅ **Replace**: Keyword matching + sentence similarity (BERT)
- **Good Enough**: 70-80% accuracy vs. 85-90% with LLMs

Behavioral Analysis:

- ❌ **Remove**: Eye tracking, facial analysis
- ✅ **Keep**: Audio analysis (speech rate, pauses, filler words)
- **Simpler Implementation**: Whisper transcription + basic audio feature extraction

Priority 3: Limit Scope to Core Features

Recommended Modules to Keep (MVP):

1. ✅ User authentication + profile
2. ✅ Resume upload + basic parsing (extract text, identify skills with keywords)
3. ✅ Template-based question generation
4. ✅ Text-based interview interaction (no real-time voice)
5. ✅ Integrated coding environment with **limited** languages (Python, JavaScript)
6. ✅ Basic code execution (test against predefined test cases)
7. ✅ Answer keyword/similarity scoring
8. ✅ Simple audio analysis (upload recording, analyze pace)
9. ✅ Transparent scoring report
10. ✅ Admin panel for question management

Modules to Defer/Remove:

- ❌ Real-time voice/video
- ❌ Advanced behavioral AI (eye tracking, etc.)
- ❌ Multi-language support (limit to 2-3 languages)
- ❌ Advanced NLP (stick to pretrained models)

8.2 Phased Development Plan

Phase 1 (Months 1-3): Core Platform

- User authentication
- Resume upload + text extraction
- Question bank management (admin panel)
- Text-based interview UI

Phase 2 (Months 4-6): Coding Assessment

- Coding environment UI (use Monaco Editor - VSCode's editor)
- Backend code execution (use Piston API - free, 5req/sec or build custom)
- Test case validation
- Basic scoring

Phase 3 (Months 7-9): AI Integration

- Resume skill extraction (spaCy NER)
- Answer evaluation (sentence-transformers)
- Audio upload + analysis (Whisper + basic features)
- Question-skill matching logic

Phase 4 (Months 10-12): Polish & Evaluation

- Report generation with visualizations
- System testing with real users
- Performance optimization
- Documentation + thesis writing

Key Milestone: By Month 6, have a **fully functional non-AI version**. AI is added as enhancement, not core dependency.

8.3 Technical Risk Mitigation

Code Execution Security

High-Risk Approach: Build custom sandbox

- ❌ Difficult to secure against all exploits
- ❌ Requires Docker/Kubernetes expertise
- ❌ Resource management is complex

Safer Alternatives:

1. **Use Piston API** (free, open-source): https://github.com/engineer-man/piston
    - 5 requests/second limit
    - 40+ languages supported
    - Battle-tested security
2. **Use Judge0 API** (50 executions/day free):
    - Professional-grade
    - Easy integration
    - **Limitation**: 50/day is low for testing
3. **Self-Host Judge0**:
    - Free, unlimited
    - Requires server (can use free tier cloud)
    - Good compromise

Recommendation: Use **Piston** for development, self-host **Judge0** for deployment

AI Model Performance

Problem: Slow inference on student hardware

Solutions:

1. **Model Quantization**: Use INT8 quantized models (2-4x faster, minimal accuracy loss)
2. **Model Distillation**: Use DistilBERT (40% smaller, 60% faster than BERT)
3. **Batch Processing**: Queue answer evaluations instead of real-time
4. **Caching**: Cache common question-answer pairs

Example Optimizations:

```
# Instead of real-time inference:
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')  # Lightweight

# Use quantization:
model = model.half()  # FP16 precision (2x faster)
```

Deployment Strategy

Development:

- Run everything locally (no cloud costs)
- Use SQLite for database (no hosting needed)
- Test with limited users (classmates)

Demo/Presentation:

- Deploy frontend to Vercel (free)
- Deploy backend to Render.com (free tier)
- Use Supabase PostgreSQL (free)
- **Accept**: 30-second cold starts, slower responses
- **Prepare**: Local demo as backup if cloud fails

8.4 Simplified Tech Stack

Original Proposed Stack: Too complex

Recommended Simplified Stack:

Layer	Technology	Reason
Frontend	React + Vite (not Next.js)	Simpler, no SSR complexity
Styling	Tailwind CSS	Keep as-is
Backend	FastAPI only (not Node.js)	Python ecosystem for AI
Database	PostgreSQL only	No need for MongoDB
AI/ML	HuggingFace Transformers	Free, pretrained models
STT	Whisper (local)	Free, accurate
TTS	Coqui TTS / gTTS	Free
Code Execution	Piston API	Free, secure
Deployment	Vercel + Render	Free tiers

Why Simpler?

- Less context switching between languages
- Easier debugging
- Faster development
- Lower hosting requirements

8.5 Scope Control Checklist

Before implementing ANY feature, ask:

1. ✅ **Is this essential for core functionality?**
2. ✅ **Can we demo the project without this?**
3. ✅ **Does this require paid services?** (If yes, find alternative)
4. ✅ **Do we have the skills to implement this?** (Be honest)
5. ✅ **Will this work on free hosting?** (Test early)

Red Flags to Avoid:

- ❌ "We'll figure out deployment later"
- ❌ "We'll use GPT-4 for now and switch later" (you won't)
- ❌ "Let's add this cool feature" (scope creep)
- ❌ "It works on my laptop" (won't work on free cloud)

---

9. Final Verdict & Recommendations

9.1 Overall Assessment

Project as Proposed: ⚠️ **NOT RECOMMENDED**

- Too ambitious for 3 students in 1 year
- High risk of incomplete deliverables
- Security concerns with code execution
- Likely to exceed zero-budget constraint during development

Project with Modifications: ✅ **RECOMMENDED WITH CHANGES**

- Strong academic value if scope is controlled
- Addresses real-world problem
- Good learning opportunities across multiple domains
- Achievable with disciplined project management

9.2 Recommended Project Scope (Revised)

Title: "AI-Assisted Mock Interview Platform with Coding Assessment"

Core Features (Must Have):

1. User registration + role-based profiles
2. Resume upload + skill extraction (keyword-based)
3. Template-based question generation (match resume skills to question bank)
4. **Text-based interview interface** (no real-time voice)
5. Integrated coding environment (Python, JavaScript only)
6. Code execution with test case validation (using Piston API)
7. Answer evaluation using sentence similarity (BERT/Sentence-Transformers)
8. Transparent scoring system (breakdown by technical, coding, communication)
9. Detailed feedback report with improvement suggestions
10. Admin panel for question/test case management

Enhanced Features (Good to Have):

1. Audio upload + speech analysis (pace, filler words) - **not real-time**
2. Multi-turn interview conversations (follow-up questions)
3. Difficulty adaptation based on performance
4. Performance analytics dashboard for admins

Explicitly Excluded:

- ❌ Real-time voice interaction
- ❌ Computer vision behavioral analysis
- ❌ Eye tracking, gaze detection
- ❌ LLM-based question generation (use templates)
- ❌ Emotion recognition
- ❌ Multi-language code support (limit to 2-3)

9.3 Resource Allocation

Ahsan Iqbal (Behavioral Analysis → Audio Analysis):

- Audio feature extraction (pace, pauses, filler words detection)
- Report generation module (PDF export with charts)
- Performance analytics dashboard
- **Estimated Effort**: 30% of project

Zain Ali Khan (Backend + AI):

- FastAPI backend architecture
- Resume parsing (PyMuPDF, spaCy)
- NLP answer evaluation (Sentence-Transformers)
- Database schema design
- API integration (Piston for code execution)
- **Estimated Effort**: 40% of project (heaviest workload)

Khizar Shahid (Frontend + Coding Module):

- React UI/UX design
- Interview interface (text-based chat)
- Coding environment integration (Monaco Editor)
- Admin panel for question management
- **Estimated Effort**: 30% of project

Shared Responsibilities:

- System testing (all)
- Documentation (all)
- Deployment (Zain + Khizar)

9.4 Success Criteria

Minimum Viable Product (MVP):

- ✅ User can upload resume and select role/difficulty
- ✅ System extracts skills and generates relevant questions
- ✅ User completes text-based interview with 5-10 questions
- ✅ User solves 2-3 coding problems with automated test cases
- ✅ System evaluates answers and code, generates score
- ✅ User receives detailed report with strengths/weaknesses

Acceptable Quality Thresholds:

- Resume parsing: 70% accuracy on skill extraction
- Answer evaluation: 65-75% agreement with human evaluators
- Code execution: 95% success rate (no crashes, timeouts handled)
- System response time: <5 seconds per question (on free hosting)
- User satisfaction: 3.5/5 stars in testing (n=20 users)

Demonstrates Technical Competence:

- Integration of multiple AI/ML models
- Secure code execution
- Full-stack web development
- Database design
- API development
- Testing and evaluation methodology

9.5 Academic Value Assessment

Research Questions the Project Can Address:

1. How effective are template-based vs. L

LM-based question generation for technical interviews? 2. Can automated answer evaluation using NLP achieve acceptable inter-rater reliability? 3. What audio features correlate with interview confidence/performance? 4. How do candidates perceive AI-based interview feedback vs. human feedback?

Potential Publications/Outcomes:

- Conference paper on automated interview systems (local CS conferences)
- Technical report on NLP-based answer evaluation
- Open-source contribution (if system is well-built)
- Useful tool for university career center

FYP Deliverables:

- Working software prototype ✅
- User study with 20-30 participants ✅
- Evaluation comparing AI scores to human ratings ✅
- Technical documentation ✅
- Final thesis/report ✅

9.6 Contingency Plans

If Behind Schedule (Month 6 Checkpoint):

1. **Drop**: Audio analysis, admin panel enhancements
2. **Keep**: Core interview + coding assessment
3. **Focus**: Polishing existing features, user testing

If Technical Blockers:

- Resume parsing too difficult → Use simpler regex-based extraction
- Answer evaluation accuracy poor → Add manual keyword lists as fallback
- Code execution issues → Use Piston API (don't build custom)
- Deployment costs arise → Demo locally, document deployment instructions

If Budget Issues Arise:

- Free tier limits exhausted → Pause development, use local testing

- GPU needed → Use Colab for experiments, CPU for deployment
- Storage limits → Delete test data regularly, use compression

---

9.7 Final Recommendation to Students

✅ PROCEED WITH MODIFIED SCOPE

Key Actions Before Starting:

1. **Week 1**: Supervisor approval of revised scope
2. **Week 2**: Set up development environment (local Python, React)
3. **Week 3**: Create detailed Gantt chart with weekly milestones
4. **Week 4**: Implement authentication + basic UI (prove you can build)
5. **Month 2**: Working text-based interview (no AI yet)
6. **Month 3**: Code execution integration
7. **Month 4-6**: Add AI components incrementally
8. **Month 7-9**: User testing + refinements
9. **Month 10-12**: Documentation, thesis writing, final polish

Success Factors:

- ✅ Weekly team meetings (mandatory)
- ✅ Bi-weekly supervisor check-ins
- ✅ Version control from Day 1 (GitHub)
- ✅ Test early, test often
- ✅ Document as you go (not at the end)
- ✅ Have a working demo by Month 6 (non-AI version)

Failure Modes to Avoid:

- ❌ Spending 3 months on perfect UI before backend works
- ❌ Trying to implement all AI features at once
- ❌ Ignoring deployment until Week 40
- ❌ Collecting custom datasets (use existing ones)
- ❌ Building custom solutions when free APIs exist

---

10. Concrete Next Steps

Immediate Actions (Next 2 Weeks):

1. **Revise Project Proposal**:
    - Remove real-time voice/video
    - Remove computer vision analysis
    - Simplify to text-based + audio upload
    - Get supervisor sign-off
2. **Technology Validation**:
    - Install Whisper, test STT locally
    - Test Piston API (code execution)
    - Run BERT inference on student laptops (measure speed)
    - Confirm free hosting works (deploy "Hello World")
3. **Dataset Acquisition**:
    - Download resume dataset from Kaggle
    - Collect interview Q&A from GitHub repos
    - Get CodeNet or similar coding dataset
    - **Do not** attempt to collect custom behavioral data
4. **Architecture Design**:
    - Draw system architecture diagram
    - Define database schema
    - Plan API endpoints
    - Decide on deployment strategy
5. **Risk Register**:
    - List top 10 risks
    - Assign mitigation strategies
    - Review monthly

---

11. Summary Table

Aspect	Rating	Key Points
Overall Feasibility	⚠️ Medium-Low (as proposed) ✅ Medium-High (revised)	Too ambitious originally; achievable with scope reduction
Technical Complexity	⚠️ High	Multiple AI domains; code execution security risk

Dataset Requirements	✅ Good availability	Q&A, coding problems available; behavioral data problematic
AI/ML Usage	✅ Feasible with pretrained models	Avoid training from scratch; use HuggingFace
Hardware Requirements	⚠️ 16GB RAM recommended	Can work on 8GB with optimization; no GPU required
Budget	✅ Zero-cost achievable	Use free tiers; avoid paid APIs; accept limitations
Paid API Dependency	✅ Avoidable	Use Whisper, Coqui TTS, open-source LLMs
Scope Control	⚠️ Critical	Remove 40% of features; focus on core MVP
Academic Value	✅ High	Strong FYP if executed well; publishable results possible
Recommendation	✅ **Proceed with modifications**	Reduce scope, phase development, test early

---

12. Final Words

This project has **strong potential** but requires **disciplined scope management**. The team seems capable (good division of labor), but the initial proposal overestimates what's achievable in one year without a budget.

The path to success:

1. Accept that "good enough" is better than "perfect but incomplete"
2. Build incrementally: text interview → coding → AI → audio
3. Use existing tools/APIs instead of reinventing
4. Test with real users early (Month 6)
5. Have a working demo at all times

Red flags to monitor:

- Week 12: Still no working prototype → major concern
- Week 20: No user testing done → revise timeline
- Week 30: Adding new features → STOP, focus on polish

With the right mindset and scope control, this can be an **excellent FYP** that demonstrates strong technical skills while remaining achievable. The key is starting simple and adding complexity gradually, not the other way around.

Good luck to Ahsan, Zain, and Khizar! 🚀